SQL interview questions - part 3

1. Aggregation and Grouping
    a. Aggregate functions (SUM, AVG, COUNT, etc.)
    b. GROUP BY clause
    c. HAVING clause
2. Subqueries and Common Table Expressions (CTEs)
    a. Correlated and non-correlated subqueries
    b. CTEs for complex queries
    c. Derived tables
3. Data Manipulation and Definition
    a. INSERT, UPDATE, DELETE statements
    b. CREATE, ALTER, DROP table operations
    c. Constraints and indexes

1. Question: How would you find the total sales for each year?

Solution:

```
SELECT EXTRACT(YEAR FROM sale_date) AS year, SUM(amount) AS
total_sales
FROM sales
GROUP BY EXTRACT(YEAR FROM sale_date)
ORDER BY year;
```

Explanation: This query extracts the year from the sale date, groups by year, and sums the sales amounts.

2. Question: Write a query to find customers who have spent more than the average order amount.

Solution:

```
SELECT customer_id, AVG(order_amount) AS avg_order
```

```
FROM orders
GROUP BY customer_id
HAVING AVG(order_amount) > (SELECT AVG(order_amount) FROM orders);
```

Explanation: This query uses a subquery to calculate the overall average order amount and compares each customer's average to this.

3. Question: How would you create a table to store customer information?

Solution:

```
CREATE TABLE customers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Explanation: This CREATE TABLE statement defines a customers table with various constraints.

4. Question: Write a query to find the second highest salary using a subquery.

Solution:

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Explanation: This query uses a subquery to find the maximum salary, then finds the maximum salary less than that.

5. Question: How would you update the email addresses of all customers in a specific city?

Solution:

```
UPDATE customers
SET email = CONCAT(LOWER(REPLACE(name, ' ', '.')), '@newdomain.com')
WHERE city = 'New York';
```

Explanation: This UPDATE statement changes email addresses for customers in New York City.

6. Question: Write a CTE to rank products by their sales volume.

Solution:

```
WITH ranked_products AS (
    SELECT product_id, SUM(quantity) AS total_sold,
           RANK() OVER (ORDER BY SUM(quantity) DESC) AS rank
    FROM sales
    GROUP BY product_id
)
SELECT * FROM ranked_products WHERE rank <= 5;
```

Explanation: This CTE ranks products by total quantity sold and selects the top 5.

7. Question: How would you add a new column to an existing table?

Solution:

```
ALTER TABLE employees
ADD COLUMN phone_number VARCHAR(15);
```

Explanation: This ALTER TABLE statement adds a new phone_number column to the employees table.

8. Question: Write a query to find departments where the average salary is higher than the company's overall average salary.

Solution:

```
SELECT department, AVG(salary) AS dept_avg
FROM employees
GROUP BY department
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

Explanation: This query compares each department's average salary to the overall company average using a subquery.

9. Question: How would you delete all orders from a specific customer?

Solution:

```
DELETE FROM orders
WHERE customer_id = 1001;
```

Explanation: This DELETE statement removes all orders associated with customer ID 1001.

10. Question: Write a query using a derived table to find the top selling product in each category.

Solution:

```
SELECT category, product_id, total_sales
FROM (
    SELECT category, product_id, SUM(amount) AS total_sales,
```

```
            RANK() OVER (PARTITION BY category ORDER BY SUM(amount)
DESC) AS rank
    FROM sales
    GROUP BY category, product_id
) AS ranked_sales
WHERE rank = 1;
```

Explanation: This query uses a derived table to rank products within categories and selects the top-ranked product for each.

11. Question: How would you create an index on the email column of the customers table?

Solution:

```
CREATE INDEX idx_email ON customers(email);
```

Explanation: This CREATE INDEX statement adds an index on the email column to improve query performance.

12. Question: Write a correlated subquery to find employees who earn more than the average salary in their department.

Solution:

```
SELECT name, salary, department
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

Explanation: This correlated subquery compares each employee's salary to their department's average.

13. Question: How would you find the total sales for each product, including products with no sales?

Solution:

```
SELECT p.product_id, p.product_name, COALESCE(SUM(s.amount), 0) AS
total_sales
FROM products p
LEFT JOIN sales s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name;
```

Explanation: This query uses a LEFT JOIN to include all products and COALESCE to replace NULL with 0 for products with no sales.

14. Question: Write a query to pivot data, showing sales amounts by product and month.

Solution:

```
SELECT product_id,
       SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 1 THEN amount
ELSE 0 END) AS Jan,
       SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 2 THEN amount
ELSE 0 END) AS Feb,
       -- ... repeat for other months
       SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 12 THEN amount
ELSE 0 END) AS Dec
FROM sales
GROUP BY product_id;
```

Explanation: This query uses conditional aggregation to pivot sales data by month.

15. Question: How would you create a view that shows the total sales for each salesperson?

Solution:

```
CREATE VIEW salesperson_performance AS
SELECT salesperson_id, name, SUM(amount) AS total_sales
FROM sales
JOIN salespeople ON sales.salesperson_id = salespeople.id
GROUP BY salesperson_id, name;
```

Explanation: This CREATE VIEW statement defines a view that summarizes sales performance.

16. Question: Write a query to find the running total of sales for each day.

Solution:

```
SELECT sale_date,
       amount,
       SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM sales
ORDER BY sale_date;
```

Explanation: This query uses a window function to calculate a running total of sales.

17. Question: How would you add a foreign key constraint to a table?

Solution:

```
ALTER TABLE orders
ADD CONSTRAINT fk_customer
FOREIGN KEY (customer_id) REFERENCES customers(id);
```

Explanation: This ALTER TABLE statement adds a foreign key constraint to the orders table, referencing the customers table.

18. Question: Write a query to find the percentage of total sales for each product category.

Solution:

```
WITH category_sales AS (
    SELECT category, SUM(amount) AS total_sales
    FROM sales
    GROUP BY category
)
SELECT category,
       total_sales,
       (total_sales * 100.0 / SUM(total_sales) OVER ()) AS percentage
FROM category_sales;
```

Explanation: This query uses a CTE and window function to calculate the percentage of total sales for each category.

19. Question: How would you insert data into a table from another table?

Solution:

```
INSERT INTO new_employees (name, department, salary)
SELECT name, department, salary
FROM old_employees
WHERE hire_date < '2020-01-01';
```

Explanation: This INSERT statement copies data from the old_employees table to the new_employees table, with a condition.

20. Question: Write a query to find the median salary for each department.

Solution:

```
WITH ranked_salaries AS (
    SELECT department, salary,
           ROW_NUMBER() OVER (PARTITION BY department ORDER BY
salary) AS row_num,
           COUNT(*) OVER (PARTITION BY department) AS dept_count
    FROM employees
)
SELECT department, AVG(salary) AS median_salary
FROM ranked_salaries
WHERE row_num IN (FLOOR((dept_count+1)/2), CEIL((dept_count+1)/2))
GROUP BY department;
```

Explanation: This query uses a CTE with window functions to calculate the median salary for each department.

21. Question: How would you create a temporary table to store intermediate results?

Solution:

```
CREATE TEMPORARY TABLE temp_results AS
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Explanation: This CREATE TEMPORARY TABLE statement creates a temporary table to store average salaries by department.

22. Question: Write a query to find employees who have worked on all projects.

Solution:

```
SELECT employee_id
FROM employee_projects
GROUP BY employee_id
HAVING COUNT(DISTINCT project_id) = (SELECT COUNT(*) FROM projects);
```

Explanation: This query compares the count of distinct projects for each employee to the total number of projects.

23. Question: How would you remove a constraint from a table?

Solution:

```
ALTER TABLE orders
DROP CONSTRAINT fk_customer;
```

Explanation: This ALTER TABLE statement removes the foreign key constraint named fk_customer from the orders table.

24. Question: Write a query to find the top 3 selling products for each month.

Solution:

```
WITH monthly_sales AS (
    SELECT EXTRACT(YEAR FROM sale_date) AS year,
           EXTRACT(MONTH FROM sale_date) AS month,
           product_id,
           SUM(amount) AS total_sales,
           RANK() OVER (PARTITION BY EXTRACT(YEAR FROM sale_date),
EXTRACT(MONTH FROM sale_date)
                        ORDER BY SUM(amount) DESC) AS rank
    FROM sales
    GROUP BY EXTRACT(YEAR FROM sale_date), EXTRACT(MONTH FROM
sale_date), product_id
)
SELECT year, month, product_id, total_sales
FROM monthly_sales
WHERE rank <= 3
ORDER BY year, month, rank;
```

Explanation: This query uses a CTE with window functions to rank products by sales for each month and selects the top 3.

25. Question: How would you create a materialized view?

Solution:

```
CREATE MATERIALIZED VIEW mv_daily_sales AS
SELECT sale_date, SUM(amount) AS total_sales
FROM sales
GROUP BY sale_date;
```

Explanation: This CREATE MATERIALIZED VIEW statement creates a materialized view of daily sales totals. Note that not all database systems support materialized views.